

Introduction to MySQL

• Databases: Collections of organized information that can be easily accessed, managed and ~~accessed~~. Stored

• An essential thing about database is to organizing data to minimize repetition.

• Database is the backbone of your application.

Why ERD first?

ERD: Entity Relationship Diagram:

- Visual blueprints for how your database looks and behaves.
- a map of the structure of how we want to store our data

SQL: is the language we use to manipulate the data as per the relationships we defined in our ERD.

Why ERD?

- ERD is laying out your tables and establishing relationships between them.
- ERD makes your data relational.

Normalization = efficient storage + flexible, customized databases with SQL

Overview

→ Main concept is Don't Repeat Data

→ Database relationships:

- one to one
- one to many
- many to many

→ 3 Forms of Normalization

→ MySQL Workbench

→ Data Types

What's the point?

• Normalization = splitting data into multiple tables to avoid repetition.

• ~~This saves data~~

• This saves storage and makes data easier to manage.

• Relationships (IDs & Foreign Keys)

Connects tables together like ~~keys~~ glue.

• With SQL you can combine data from different tables to make custom views for specific tasks.

Normalization: the process of splitting one big table into smaller, related tables, so we don't repeat data.

• Each table focuses on one job.

• By splitting tables:

• Database becomes Modular (flexible, easy to expand)

• We can customize tables depending on ~~needs~~

One to One Relationship

Customers table:

id	First name	last name	address-id
		--	4
		--	5
			6

This is called a Foreign Key for addresses table. This is how we are able to join the addresses together with the customers.

addresses table:

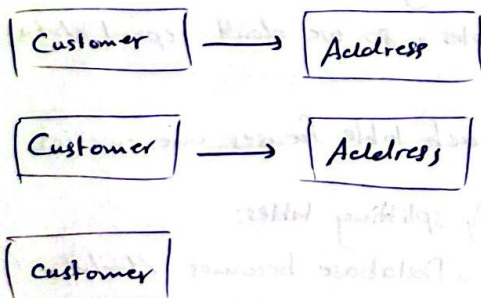
id	Street	City	State
4			
5			
6			

Since each address belongs to one customer and each customer can have only one address, So this is

One to One Relationship

NOTE A relationship does not necessarily exist just because we have one of the pieces.

⇒ means that a relationship doesn't exist for a customer



Examples of One-to-one:

- Customers and credit cards:

every customer has one credit card
every credit card belongs to one customer

- User and email:

every user have one email address
every email has one user

- Product and Image:

every product has an image
and every image is of a product

⇒ Person to Passport , Country to Capital city

NOTES

How SQL Joins work.

- Databases are split into tables by the db designer
- To connect those tables we use relationships.
- Relationships are built using:

Primary Key (ID) → unique identifier in one table

Foreign Key → a column in another table that refers to that ID.

One - to - Many
Many - to - One

Many - to - Many

orders:

id	customer-id
687	1
688	2
690	2
689	3

items:

id	name	description
110		
111		
112		
113		

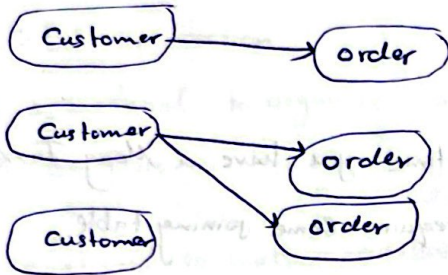
relationship: orders can have many items,
and items can have many orders

NOTE In many-to-many, we create
a connector table with both order-id
and the item-id,

items - orders

order-id	item-id
687	110
687	111
688	112
688	115
689	113
689	114
689	115
690	113

- With the previous addresses/customers example
- We'll add orders table, each customer has multiple orders, but each order can belong to one Customer only.



SOL

Customers			
id	first-name	last-name	address-id
1			
2			
3			

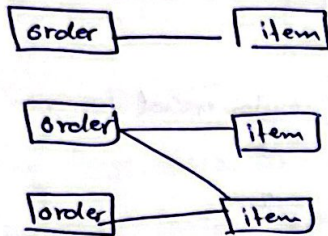
Orders

id	order-date	amount	customer-id
687			
688			
689			

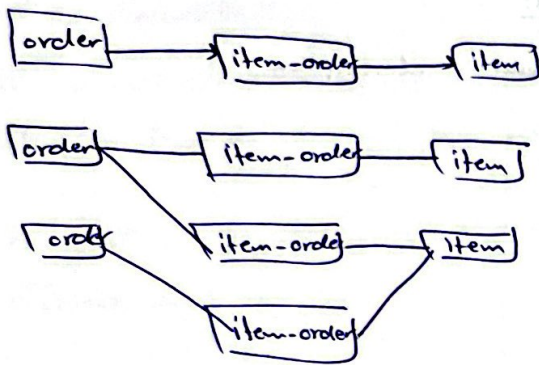
Examples:

- Messages and comments
- States and cities
- Customers and orders
- ~~Teacher and students~~
- Parent and child

items - orders:



graph:



SQL

orders:

id	order-date	amount	customer-id
687			
688			
689			

items:

id	name	description
110		
111		
112		

orders - items:

orders-id	items-id
687	110
687	110
688	112

Examples:

• Users & Interests

• Actors - Movies

• Businesses ~~Area~~ - Cities

one business can be spread across multiple cities

and one city can have many businesses

NOTE anytime you have a Many-To-Many it will require some joining table

Normalization

Database: normalization: is a convention for splitting large data ~~to~~ tables into smaller separate tables with the primary goal of not repeating data.

What is Normalization:

- it's a method to organize data into tables so there is no repeating data.
- The goal is: to make database more efficient and consistent.
- example instead of storing the same email in many places, store it ~~is~~ once and link it with an ID.

before Normalization:

Student ID	Name	Course	Instructor	Instr email
------------	------	--------	------------	-------------

Problem:

- Instructor info is repeated many times
- if Dr. Ali changes email, we must update it everywhere.
- Table is large, messy, and inefficient.

Step 1: Split into Smaller groups:

- Students table St. ID | Name
- Course table CourseID | Course Name | Instructor ID
- Instructor table - -

The 3 Forms of Normalization:

1. First normal form (1NF)

Rule: each column should store one value only

Wrong: address = "gaza, palestine, 007"

Correct: split into City, Country, Zipcode

2. Second normal form (2NF)

Rule: Columns must store unique values (except keys)

Wrong: Movies table, having Category = Action written many times

Correct: create a Categories table, and use a foreign Key to connect

3. Third Normal Form (3NF)

Rule: A column should not depend on another non Key column.

Wrong: In a Books table with publisher Name - publisher Address, the address depends on the name (both are not keys)

Correct: Move publisher info into separate publishers table, and link it with a foreign Key.

Example on (2NF)

X Not in 2NF:

StudentID	CourseID	StudentName
-----------	----------	-------------

- primary Key = (StudentID, CourseID)
- but StudentName depends only on StudentID, not on the whole Key.
- This causes repetition of Student names.

Fixed:

StudentID	StudentName
-----------	-------------

StudentID	CourseID
-----------	----------

(1NF) → each column in your table can only have one value

(2NF) → each column in the table is not a Key (primary or foreign) must have unique values

(3NF) → you cannot have a non-Key column that is dependent on another non-Key column.

Conventions

* Guidelines for Conventions:

1. Make the table name plural and All lowercase
make it plural (users, leads, sites, clients,
chapters, courses, modules)

2. Use "id" as the primary Key
name it id, make it auto incremented.

3. Name Foreign Keys with singular-table-name-id
which ~~referring~~ referencing a primary Key in
another table.

→ When you name a Foreign Key (a column
that points to another table's id) you should
name it $\Rightarrow$ singular-table-name-id

users table

id name

orders table

id order-date user-id

4. Use created_at, updated_at as columns
for the timestamp in Every table you create.

NOTES

• Primary Key (id):

→ unique identifier for each row
in the table

→ in users table, id uniquely
identifies each user.

• Foreign Key (*-id):

→ Points to another table's id
(or sometimes the same table's id
if it's a self-relationship)

→ in orders table, user-id points
to users.id.

Main purpose of Normalization

Creating convention for organizing
and avoiding duplication of data

Data Types

• VARCHAR: number of characters:

A-Z
for english only

nvarchar(50)
↓
تعداد

- They store non-numeric values up to 255 characters.
- Used for values with different length (email / username / last name)
characters
- A VARCHAR is called a variable number of characters because it can store a variable number of characters and only uses the space required for each record stored in the DB.
- VARCHAR = variable character
means the storage size changes depending on the length of the text you put in

• Ex VARCHAR(10)

"Hi" → only 2

"Noor" → only 4

"programming" → >10 not allowed

• CHAR: number of characters:

- Used for non-numeric
- It will use up all space for the set number of characters regardless of the actual val
- Good to use for things that will always be given number of characters
- ex CHAR(15)

Storing "Coding" it will use up the equivalent of 15 characters

• FLOAT

- for floating-point numbers
- ex item-cost

• INT (used for integers):

- Used to store whole numbers (no decimals)
- Used for ID's like (user-id, order-id)

how it works:

- An INT can hold up to about 2.1 billion values $\rightarrow$ which is usually more than enough for most tables
- good for $\rightarrow$ phone numbers, zip-codes

Signed vs. Unsigned

- Unsigned $\rightarrow$ only positive numbers from (0 $\rightarrow$ 4.2 billion)
- Signed $\rightarrow$ can be both negative / positive from (-2.1 billion $\rightarrow$ +2.1 billion)

NOTE if you don't need negative numbers:

use UNSIGNED to double the positive range.

• BIGINT

- Used for huge numbers like facebook ID's

$\rightarrow$ unsigned 18.44 quintillion

$\rightarrow$ signed -9.22 quintillion $\rightarrow$ +9.22 quintillion

• TINYINT

- Used for relatively small numbers

• ex user_level identifier (0-inactive user, 1-active user, 9-admin)

• unsigned $\rightarrow$ 0 $\rightarrow$ 255

• signed $\rightarrow$ -128 $\rightarrow$ 127

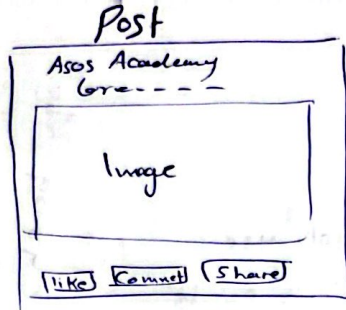
• TEXT

- Used for any text that the VARCHAR is too small for
- description, messages, comments

• DATETIME

date and time in the format $\rightarrow$ YYYY-MM-DD hh:mm:ss

Likes



- 1 Determine which database tables you need for this post
- 2 Set up the relationships
- 3 Make sure to rename all foreign keys $\Rightarrow$ singular

- users
 - id (PK)
 - name, email, password
 - created-at

- Posts
 - id (PK)
 - user-id (FK $\rightarrow$ users.id) $\Leftarrow$ singular
 - body, text, image-url

• Likes

- id (PK) / composite (PK)
- user-id (FK $\rightarrow$ users.id) $\Leftarrow$ singular
- post-id (FK $\rightarrow$ posts.id) $\Leftarrow$ singular
- created-at

Relationships:

• user 1 $\rightarrow$ ∞ post

• user $\infty \rightarrow \infty$ post via likes

$\hookrightarrow$ implemented as: User 1 $\rightarrow \infty$ like and post 1 $\rightarrow \infty$ like

Food Reviews

"users"

- id (PK)
- name
- email
- created-at

• restaurants

- id (PK)
- name
- address
- city
- phone
- image-url
- created-at

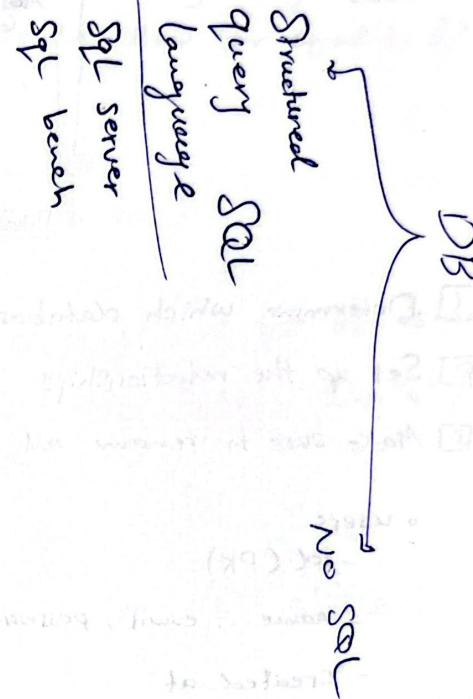
reviews

- id (PK)
- user-id (FK $\rightarrow$ users.id)
- restaurant-id (FK $\rightarrow$ rest.id)
- body
- created-id

name nvarchar

id number nvarchar

bc we'll not use
math operations on it
so we don't have to
put it INT



Primary Key

① auto increment

② Not null

③ unique

INT
VARCHAR
inc.

Table		
Columns	data types properties	properties